Depuración de texto y persistencia de datos con Python

Actividad integradora de las unidades 3 y 4

Docente: Dr. Ramón Osvaldo Guardado Medina
Programa: Maestría en Ciencia de Datos
Modalidad sugerida: parejas · Duración: 180 minutos

Integrantes: Sergio Delgadillo Domínguez
Grupo y fecha: 9 Octubre 2026

Propósito de la actividad

Construyan un programa reproducible que transforme solicitudes de soporte almacenadas como texto en registros normalizados y verificables. Aplicarán las operaciones de cadenas y el manejo de archivos de las unidades 3 y 4 para justificar decisiones de calidad de datos antes de un análisis posterior.

Situación a resolver

Una mesa de ayuda recibe solicitudes con espacios, categorías escritas de distintas formas y registros incompletos. El equipo de Ciencia de Datos necesita conocer la distribución real de las solicitudes sin que las variaciones de escritura alteren los conteos. Su tarea consiste en conservar las entradas, separar rechazos y producir resultados que otra persona pueda revisar.

Resultados de aprendizaje

- Aplicar índices, slices, búsqueda y normalización sin confundir una cadena original con su versión transformada.
- Definir reglas explícitas de calidad y documentar el efecto de la limpieza sobre los conteos.
- Procesar archivos por línea con with y UTF-8, seleccionar los modos r, w y a y manejar errores específicos.
- Verificar archivos persistidos y explicar los límites del procedimiento.

Prerrequisitos: variables, condicionales, ciclos, listas y diccionarios. Se permite consultar las presentaciones. Utilicen la biblioteca estándar de Python; no requieren pandas ni modelos de aprendizaje automático.

Materiales y organización

Archivo incluido	Uso
datos/solicitudes.txt	Entrada del reto con encabezado y errores intencionales.
datos/etiquetas.txt	Nueve variantes para ensayar normalización.
datos/eventos.txt	Bitácora de seis líneas para el ejemplo guiado.
ejemplo_guiado.py	Demostración resuelta de cadenas y archivos.
actividad_inicio.py	Plantilla con TODO para resolver el reto.
LEEME.txt	Preparación y comandos de ejecución.

Extraigan el ZIP completo y abran su carpeta en VS Code u otro IDE. Ejecuten python ejemplo_guiado.py. Comprueben la salida: Sensor norte, Valor: 23.5 y Errores: 2. El script genera una carpeta salidas_demo.

Etapa	Minutos	Evidencia
1 Diagnóstico de cadenas	25	Predicciones y comparación de etiquetas.
2 Ejemplo guiado	20	Ejecución y explicación de r, w y a.
3 Depuración del archivo	65	Flujo de validación y registro de rechazos.
4 Persistencia y verificación	40	Salidas, resumen y pruebas.
5 Interpretación y cierre	30	Conclusiones y entrega.

Etapa 1 Diagnóstico de cadenas

Antes de programar, predigan los resultados para codigo = " mx-gdl-0042 ": len(codigo), codigo[2:4], codigo[-6:-2], codigo.strip().upper() y codigo.find("xyz"). Después ejecútenlos y expliquen cualquier diferencia.

```
codigo = "  mx-gdl-0042  "
print(len(codigo))              # 15
print(codigo[2:4])              # mx
print(codigo[-6:-2])            # 0042
print(codigo.strip().upper())   # MX-GDL-0042
print(codigo.find("xyz"))       # -1
```

Respondan:

¿codigo.strip() modifica codigo si no se reasigna?

Respuesta: no modifica código, si el cambio que se aplico al final no se reasigna.

¿Qué diferencia existe entre in y find()?

Respuesta: in sirve para saber si un texto esta contenido en otro, nos retorna true/false, mientras que find() sirve para obtener la posición donde se encuentra ese texto o devuelve -1 si no aparece.

¿Por qué un resultado -1 no debe utilizarse como posición encontrada?

Respuesta: porque en Python cadena[-1] significa el ultimo carácter de esa cadena, lo cual es incorrecto porque "xyz" no fue encontrado, es decir llego al final de la cadena y no lo encontró, y como llego al final retorna el indica del final de la cadena, es decir "-1".

Lean etiquetas.txt. Cuenten categorías distintas antes de limpiar, eliminando únicamente el salto de línea. Luego normalicen y vuelvan a contar. Propongan un caso en el que capitalize() alteraría una sigla que debe conservarse.

Antes de limpiar son: 9 categorías distintas, después de normalizar: 3.

Un caso en el que capitalize() alteraría una sigla que debe conservarse: "NASA".capitalize() #Nasa

Contrato de datos y reglas de validación

solicitudes.txt contiene una cabecera y 30 líneas de datos, incluida una línea vacía. Cada registro usa cuatro campos separados por |. En este archivo no aparecen separadores ni saltos de línea dentro de los campos; por eso puede utilizarse split("|"). Un CSV general con comillas requiere un analizador como csv.reader.

Campo	Regla para aceptar y transformar
id	Aplicar strip() y upper(). Exigir SOL- seguido de cuatro dígitos ASCII 0–9, longitud total 8. El ID debe ser único entre los registros aceptados.
correo	Aplicar strip() y lower(). Exigir una sola @, partes no vacías, ningún espacio y un punto en el dominio que no sea su primer ni último carácter. Es una validación didáctica, no una validación completa de correo.
categoria	Aplicar strip() y lower(); sustituir - y _ por espacios; separar palabras y unir con _. Conservar tildes. Aceptar acceso, pago_en_línea o soporte_técnico.
descripcion	Eliminar espacios extremos y reducir espacios internos consecutivos a uno. Exigir texto no vacío. Conservar mayúsculas y tildes.

Orden de evaluación obligatorio

Evalúen cada línea en este orden y registren solo el primer motivo de rechazo: línea vacía; cantidad de campos distinta de cuatro; algún campo vacío tras strip(); ID con formato inválido; correo inválido; categoría fuera del catálogo; ID duplicado. Agreguen un ID al conjunto de vistos únicamente cuando acepten el registro completo.

Numeración y trazabilidad

La cabecera ocupa la línea física 1. Numérenlas con enumerate(archivo, start=2) después de leerla. Un rechazo debe guardar el número de línea, el motivo y el texto original sin el salto final. No omitan la línea vacía: también representa un problema de calidad.

Etapa 2 Ejemplo guiado

Revisen ejemplo_guiado.py y ejecútenlo dos veces. Expliquen por qué errores.txt conserva dos líneas y ejecuciones.log aumenta. Identifiquen dónde se usa strip(), split(), replace(), capitalize(), una f-string y el cierre automático de archivos.

Porque encuentra dos errores y el archivo errores, se abre en modo “w” este modo borra el contenido previo y solo conserva lo nuevo y cada que ejecuta el script.py, se escribe el archivo “ejecuciones.log” en modo “a” que es el append, solo agrega una línea al log de los errores encontrados. etiqueta.strip() quita espacios al inicio y al final, etiqueta.strip().split(":") separa ”sensor_norte:23.5” en nombre y valor, nombre.replace("_", " ") convierte” “sensor_norte” en “sensor norte”, nombre.replace("_", " ").capitalize() convierte "sensor norte" en "Sensor norte", f-string: print(f"Valor: {float(valor):.1f}") formatea el numero con un decimal, historial.write(f"Errores detectados: {total}\n") y el cierre automático de archivos:

with (BASE / "datos" / "eventos.txt").open(...) as entrada:

with (SALIDA / "errores.txt").open("w", ...) as salida:

with (SALIDA / "ejecuciones.log").open("a", ...) as historial:

El bloque with garantiza que los archivos se cierran automáticamente, incluso si ocurre un error.

Etapa 3 Implementación del depurador

Completen actividad_inicio.py. Dividan el trabajo en funciones y documenten sus entradas y salidas. Usen el directorio del script como base para que las rutas no dependan de la carpeta desde la que ejecutan Python.

1. Comprueben el encabezado exacto id|correo|categoria|descripcion. Si no coincide, detengan el proceso con un mensaje claro antes de reemplazar resultados anteriores.

```
with ENTRADA.open(encoding="utf-8") as entrada:
    # 1. Verificar encabezado exacto
    encabezado = entrada.readline().rstrip("\n")
    esperado = "id|correo|categoria|descripcion"
```

2. Lean las solicitudes mediante iteración sobre el archivo. No usen read() ni readlines() para cargar toda la entrada. Mantengan contadores, categorías y el conjunto de IDs aceptados.

```
# 2. Iteración línea por línea (sin read/readlines)
for num, linea in enumerate(entrada, start=2):
    texto_original = linea.rstrip("\n")
    total += 1
```

3. Normalicen cada campo y apliquen las reglas en el orden establecido. Escriban un registro válido o un rechazo por cada línea de datos.

```
def normalizar_categoria(texto):
    # TODO: strip, lower, sustituir guiones/guiones bajos y unir palabras.
    #raise NotImplementedError("Completa normalizar_categoria")
    """
    Normaliza la categoría:
    - strip()
    - lower()
    - reemplaza '-' y '_' por espacios
    - une con guion bajo
    """
    limpio = texto.strip().lower().replace("-", " ").replace("_", " ")
    return "_".join(limpio.split())
```

4. Capturen FileNotFoundError con un mensaje que identifique el archivo. No utilicen except: sin tipo ni conviertan errores de programación en rechazos silenciosos.

```
except FileNotFoundError as e:
    print("Archivo no encontrado:", e.filename)
```

Rechazos.jsonl(modo w)

Cada línea contiene un objeto JSON con:

Json

```
{
  "linea": <número>,
  "motivo": "<motivo>",
  "texto_original": "<línea original>"
}
```

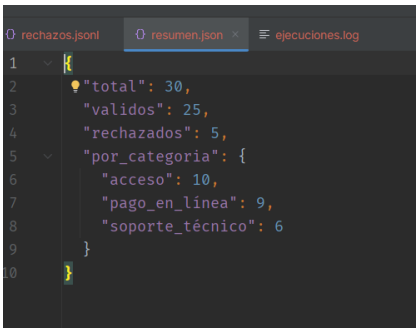
rechazos.jsonl | resumen.json | ejecuciones.log

```
{"linea": 26, "motivo": "Línea vacía", "texto_original": ""}
{"linea": 27, "motivo": "Correo inválido", "texto_original": "SOL-0025|sin_arroba.example.com|Acceso|No puedo entrar"}
{"linea": 29, "motivo": "Categoría vacía", "texto_original": "SOL-0027|alumno27@cucea.udg.mx||Falta categoría"}
{"linea": 30, "motivo": "Número incorrecto de campos", "texto_original": "SOL-0028|alumno28@cucea.udg.mx|Acceso"}
{"linea": 31, "motivo": "ID duplicado", "texto_original": "SOL-0001|otro@cucea.udg.mx|Acceso|Solicitud repetida"}
```

Resumen.json(modo w)

Contiene:





Etapa 4 Persistencia y verificación

Generen la carpeta salidas y los siguientes archivos. Conserven intacta la carpeta datos. Utilicen with y encoding="utf-8" en cada apertura de texto.

Salida	Contenido y modo
solicitudes_limpias.txt	Modo w. Encabezado original y registros válidos en orden de entrada, con el mismo separador |.
rechazos.jsonl	Modo w. Un objeto JSON por línea con linea, motivo y texto_original. Usen json.dumps(..., ensure_ascii=False).
resumen.json	Modo w. Objeto con total, validos, rechazados y por_categoria, calculados a partir de la ejecución. Usen json.dump(..., ensure_ascii=False, indent=2).
ejecuciones.log	Modo a. Una línea por ejecución exitosa, con fecha y hora, nombre de la entrada y conteos.

Reabran los resultados: cuenten los registros limpios excluyendo la cabecera, carguen el resumen con json.load() y validen cada línea de rechazos con json.loads(). Comparen lo persistido con los contadores. No basta con mostrar los valores en pantalla.

Extensión opcional: comparen la memoria de read() con la iteración por líneas en un archivo grande generado por ustedes. Consideren que el conjunto de IDs vistos también crece con la cantidad de registros únicos.

```python
# Reabrir y verificar persistencia
with limpias_path.open(encoding="utf-8") as f:
    registros_limpios = len(f.readlines()) - 1  # sin cabecera

with resumen_path.open(encoding="utf-8") as f:
    resumen_cargado = json.load(f)

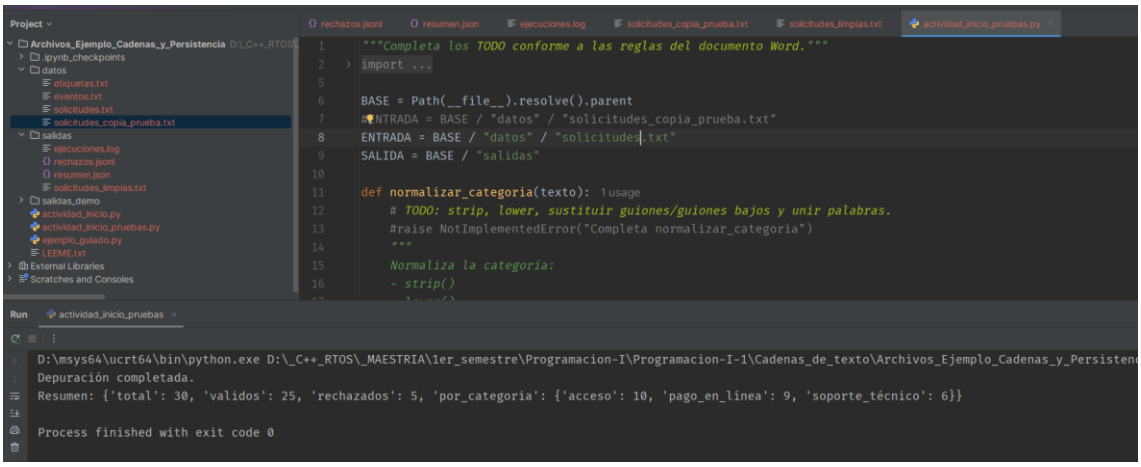
with rechazos_path.open(encoding="utf-8") as f:
    for linea in f:
        json.loads(linea)  # valida que cada línea sea JSON
```

```python
assert resumen_cargado["total"] == resumen_cargado["validos"] + resumen_cargado["rechazados"]
assert sum(resumen_cargado["por_categoria"].values()) == resumen_cargado["validos"]
assert registros_limpios == resumen_cargado["validos"]
```

Controles e interpretación de resultados

Control sobre los archivos originales	Resultado esperado
Líneas de datos sin cabecera, incluida la vacía	30
Registros válidos / rechazados	24 / 6
Solicitudes por categoría normalizada	acceso: 9 · pago_en_línea: 9 · soporte_técnico: 6
Categorías distintas en etiquetas.txt antes / después	9 / 3
Líneas físicas rechazadas	26, 27, 28, 29, 30 y 31
Tasa de rechazo y tasa de aceptación	20 % y 80 %

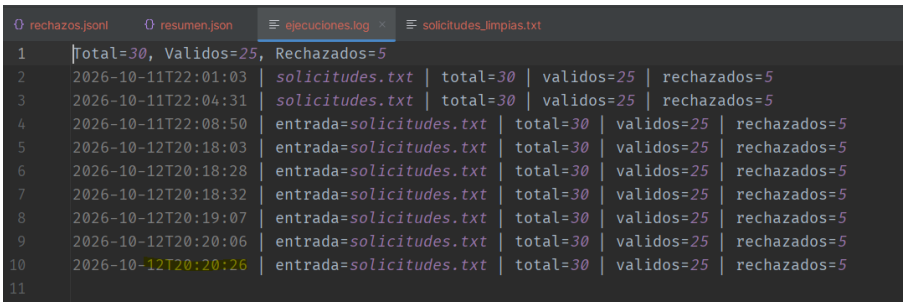
Los controles sirven para verificar su implementación; los resultados deben calcularse y no escribirse como constantes. Demuestren total = validos + rechazados y suma(por_categoria.values()) = validos.

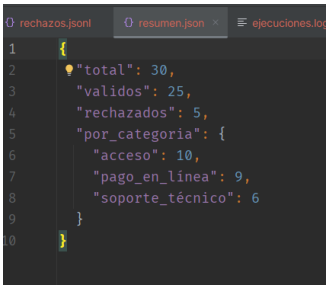


Pruebas mínimas con evidencia

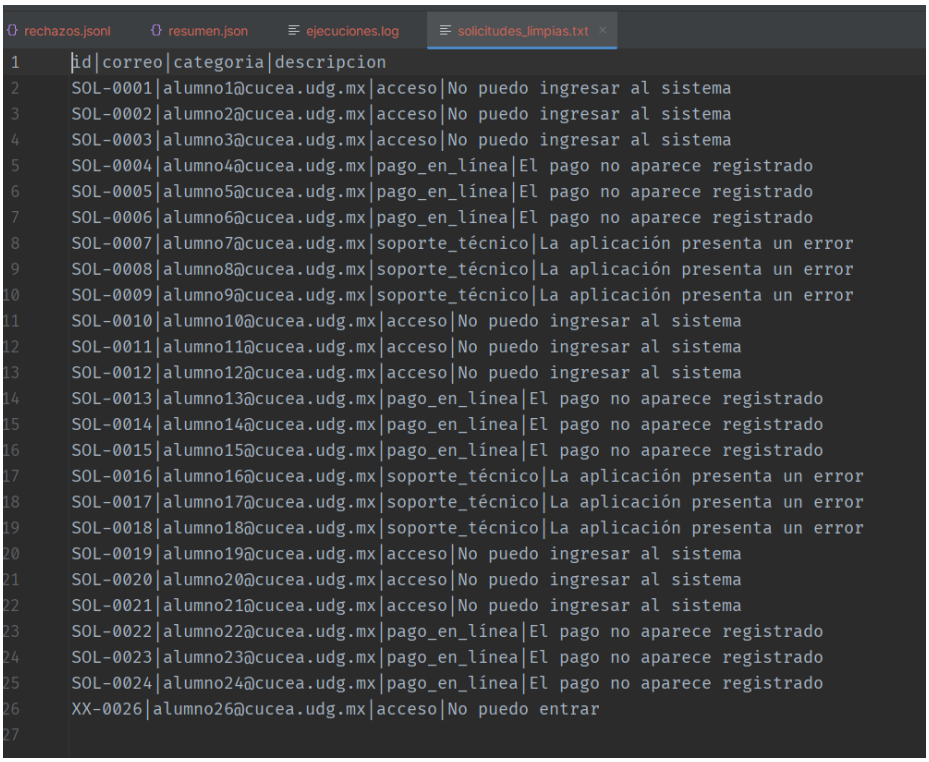
1. Ejecuten dos veces: las salidas de datos deben conservar sus conteos y el historial aumentar una línea.

Primera ejecución:

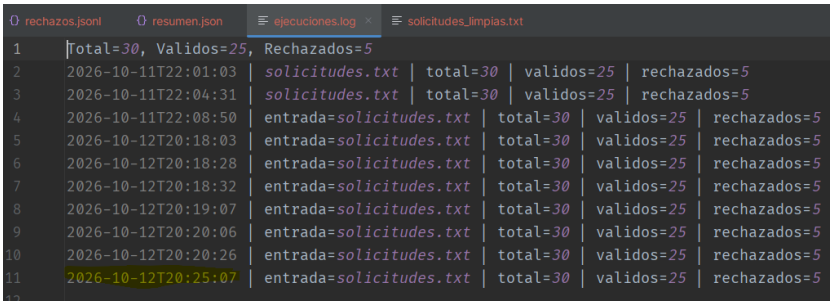




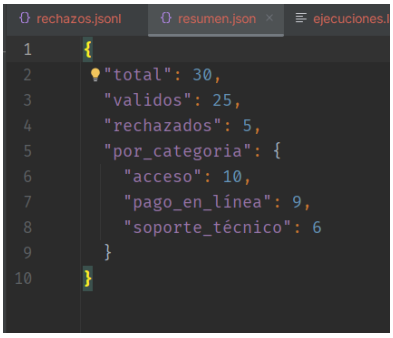




Segunda ejecución:



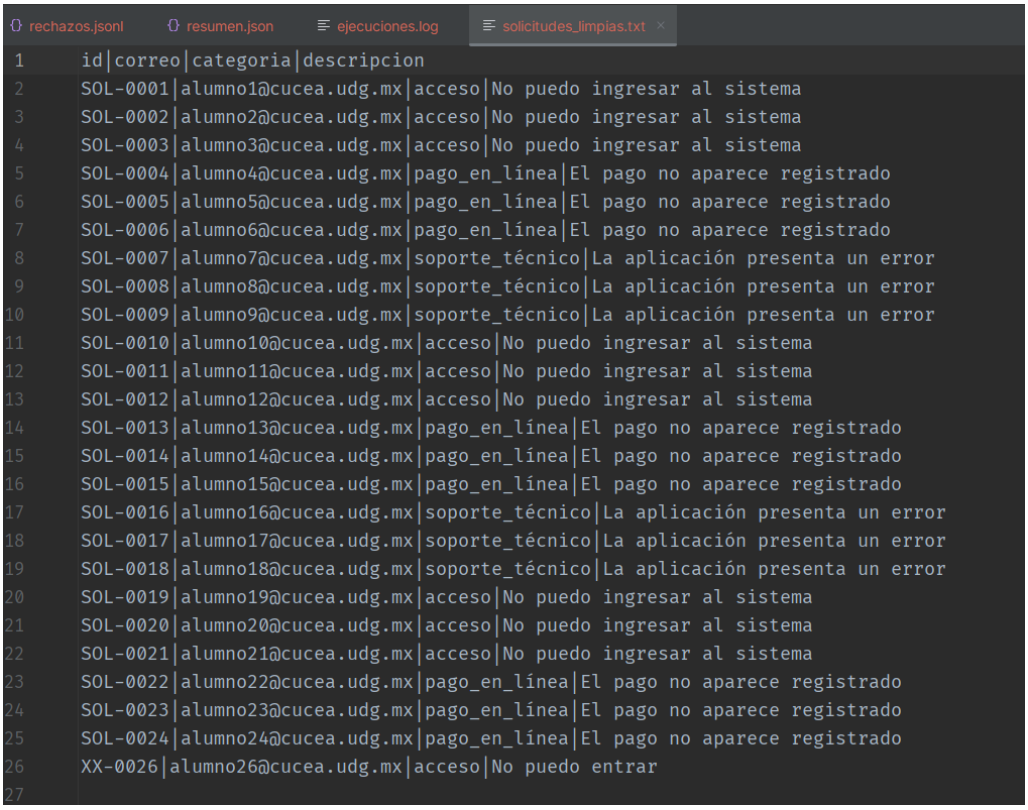
```
Total=30, Validos=25, Rechazados=5
2026-10-11T22:01:03 | solicitudes.txt | total=30 | validos=25 | rechazados=5
2026-10-11T22:04:31 | solicitudes.txt | total=30 | validos=25 | rechazados=5
2026-10-11T22:08:50 | entrada=solicitudes.txt | total=30 | validos=25 | rechazados=5
2026-10-12T20:18:03 | entrada=solicitudes.txt | total=30 | validos=25 | rechazados=5
2026-10-12T20:18:28 | entrada=solicitudes.txt | total=30 | validos=25 | rechazados=5
2026-10-12T20:18:32 | entrada=solicitudes.txt | total=30 | validos=25 | rechazados=5
2026-10-12T20:19:07 | entrada=solicitudes.txt | total=30 | validos=25 | rechazados=5
2026-10-12T20:20:06 | entrada=solicitudes.txt | total=30 | validos=25 | rechazados=5
2026-10-12T20:20:26 | entrada=solicitudes.txt | total=30 | validos=25 | rechazados=5
2026-10-12T20:25:07 | entrada=solicitudes.txt | total=30 | validos=25 | rechazados=5
```



```
{
  "total": 30,
  "validos": 25,
  "rechazados": 5,
  "por_categoria": {
    "acceso": 10,
    "pago_en_línea": 9,
    "soporte_técnico": 6
  }
}
```



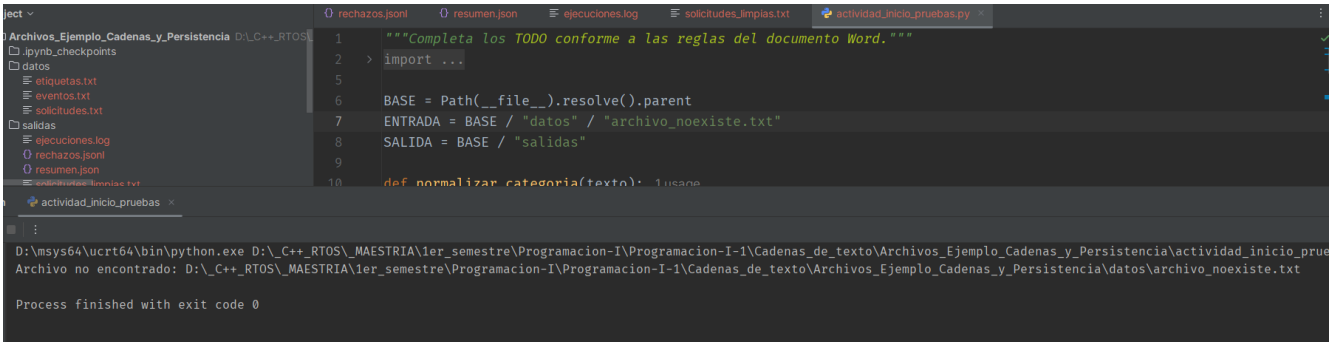
```
{"linea": 26, "motivo": "Línea vacía", "texto_original": ""}
{"linea": 27, "motivo": "Correo inválido", "texto_original": "SOL-0025|sin_arroba.example.com|Acceso|No puedo entrar"}
{"linea": 29, "motivo": "Categoría vacía", "texto_original": "SOL-0027|alumno27@cucea.udg.mx||Falta categoría"}
{"linea": 30, "motivo": "Número incorrecto de campos", "texto_original": "SOL-0028|alumno28@cucea.udg.mx|Acceso"}
{"linea": 31, "motivo": "ID duplicado", "texto_original": "SOL-0001|otro@cucea.udg.mx|Acceso|Solicitud repetida"}
```



```
id|correo|categoria|descripcion
SOL-0001|alumno1@cucea.udg.mx|acceso|No puedo ingresar al sistema
SOL-0002|alumno2@cucea.udg.mx|acceso|No puedo ingresar al sistema
SOL-0003|alumno3@cucea.udg.mx|acceso|No puedo ingresar al sistema
SOL-0004|alumno4@cucea.udg.mx|pago_en_línea|El pago no aparece registrado
SOL-0005|alumno5@cucea.udg.mx|pago_en_línea|El pago no aparece registrado
SOL-0006|alumno6@cucea.udg.mx|pago_en_línea|El pago no aparece registrado
SOL-0007|alumno7@cucea.udg.mx|soporte_técnico|La aplicación presenta un error
SOL-0008|alumno8@cucea.udg.mx|soporte_técnico|La aplicación presenta un error
SOL-0009|alumno9@cucea.udg.mx|soporte_técnico|La aplicación presenta un error
SOL-0010|alumno10@cucea.udg.mx|acceso|No puedo ingresar al sistema
SOL-0011|alumno11@cucea.udg.mx|acceso|No puedo ingresar al sistema
SOL-0012|alumno12@cucea.udg.mx|acceso|No puedo ingresar al sistema
SOL-0013|alumno13@cucea.udg.mx|pago_en_línea|El pago no aparece registrado
SOL-0014|alumno14@cucea.udg.mx|pago_en_línea|El pago no aparece registrado
SOL-0015|alumno15@cucea.udg.mx|pago_en_línea|El pago no aparece registrado
SOL-0016|alumno16@cucea.udg.mx|soporte_técnico|La aplicación presenta un error
SOL-0017|alumno17@cucea.udg.mx|soporte_técnico|La aplicación presenta un error
SOL-0018|alumno18@cucea.udg.mx|soporte_técnico|La aplicación presenta un error
SOL-0019|alumno19@cucea.udg.mx|acceso|No puedo ingresar al sistema
SOL-0020|alumno20@cucea.udg.mx|acceso|No puedo ingresar al sistema
SOL-0021|alumno21@cucea.udg.mx|acceso|No puedo ingresar al sistema
SOL-0022|alumno22@cucea.udg.mx|pago_en_línea|El pago no aparece registrado
SOL-0023|alumno23@cucea.udg.mx|pago_en_línea|El pago no aparece registrado
SOL-0024|alumno24@cucea.udg.mx|pago_en_línea|El pago no aparece registrado
XX-0026|alumno26@cucea.udg.mx|acceso|No puedo entrar
```

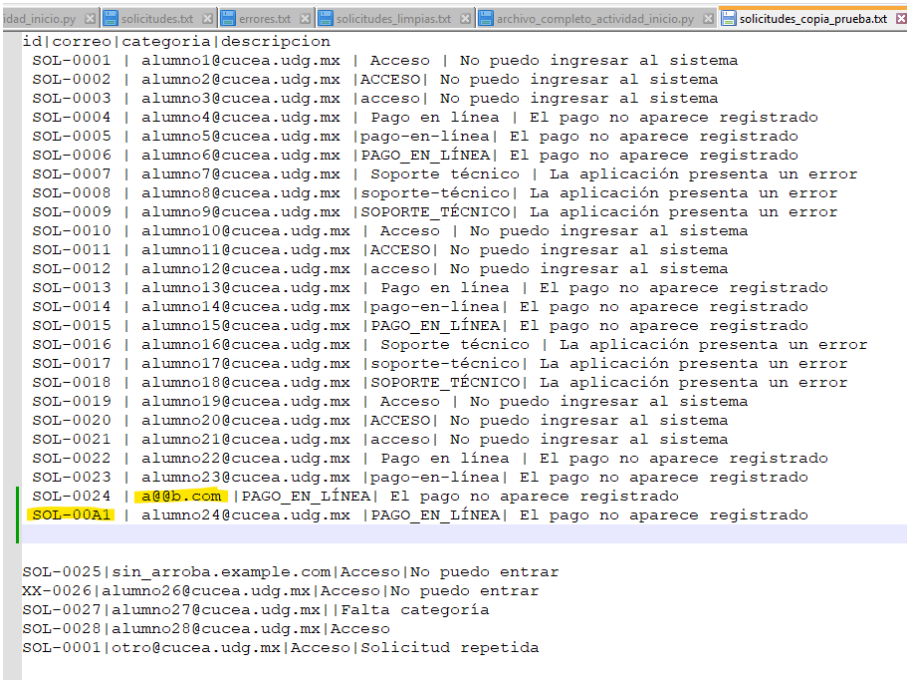
2. Prueben una ruta inexistente.

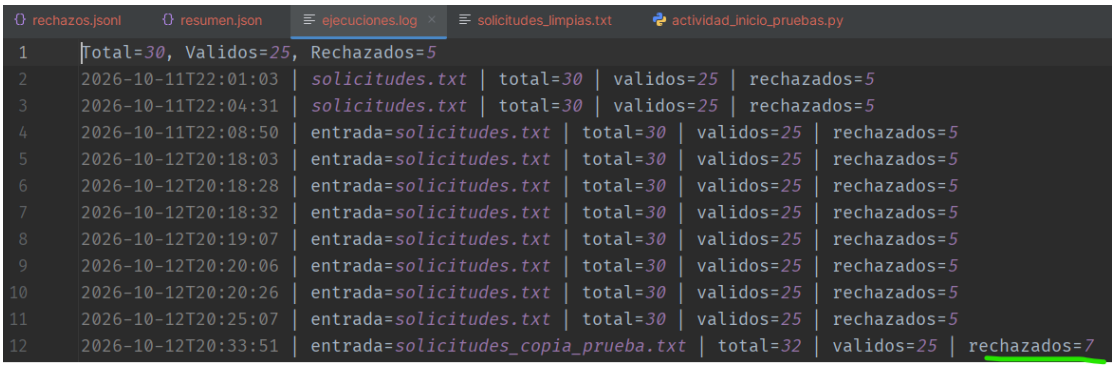
Evidencia:



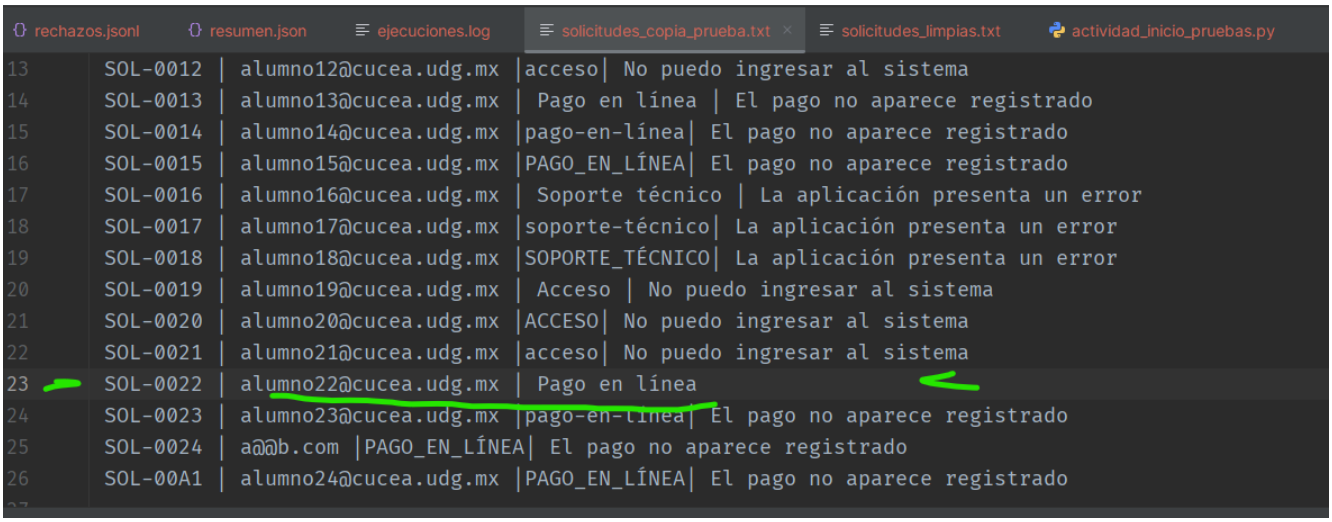
3. En una copia de prueba, agreguen un ID SOL-00A1 y un correo a@@b.com en registros distintos.

Evidencia:

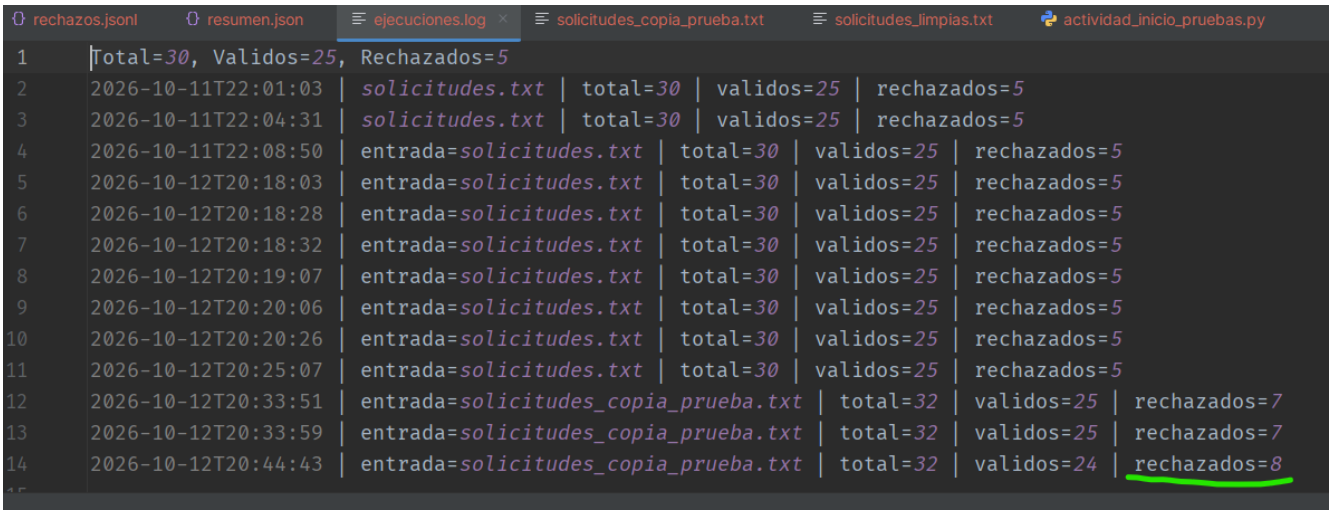




4. Prueben una cabecera incorrecta. Registren el resultado esperado y el observado; no alteren la entrada original.



```
rechazos.jsonl  resumen.json  ejecuciones.log  solicitudes_copia_prueba.txt  solicitudes_limpias.txt  actividad_inicio_pruebas.py
13  SOL-0012 | alumno12@cucea.udg.mx |acceso| No puedo ingresar al sistema
14  SOL-0013 | alumno13@cucea.udg.mx | Pago en línea | El pago no aparece registrado
15  SOL-0014 | alumno14@cucea.udg.mx |pago-en-línea| El pago no aparece registrado
16  SOL-0015 | alumno15@cucea.udg.mx |PAGO_EN_LÍNEA| El pago no aparece registrado
17  SOL-0016 | alumno16@cucea.udg.mx | Soporte técnico | La aplicación presenta un error
18  SOL-0017 | alumno17@cucea.udg.mx |soporte-técnico| La aplicación presenta un error
19  SOL-0018 | alumno18@cucea.udg.mx |SOPORTE_TÉCNICO| La aplicación presenta un error
20  SOL-0019 | alumno19@cucea.udg.mx | Acceso | No puedo ingresar al sistema
21  SOL-0020 | alumno20@cucea.udg.mx |ACCESO| No puedo ingresar al sistema
22  SOL-0021 | alumno21@cucea.udg.mx |acceso| No puedo ingresar al sistema
23  SOL-0022 | alumno22@cucea.udg.mx | Pago en línea
24  SOL-0023 | alumno23@cucea.udg.mx |pago-en-línea| El pago no aparece registrado
25  SOL-0024 | a@@b.com |PAGO_EN_LÍNEA| El pago no aparece registrado
26  SOL-00A1 | alumno24@cucea.udg.mx |PAGO_EN_LÍNEA| El pago no aparece registrado
```



```
rechazos.jsonl  resumen.json  ejecuciones.log  solicitudes_copia_prueba.txt  solicitudes_limpias.txt  actividad_inicio_pruebas.py
1   Total=30, Validos=25, Rechazados=5
2   2026-10-11T22:01:03 | solicitudes.txt | total=30 | validos=25 | rechazados=5
3   2026-10-11T22:04:31 | solicitudes.txt | total=30 | validos=25 | rechazados=5
4   2026-10-11T22:08:50 | entrada=solicitudes.txt | total=30 | validos=25 | rechazados=5
5   2026-10-12T20:18:03 | entrada=solicitudes.txt | total=30 | validos=25 | rechazados=5
6   2026-10-12T20:18:28 | entrada=solicitudes.txt | total=30 | validos=25 | rechazados=5
7   2026-10-12T20:18:32 | entrada=solicitudes.txt | total=30 | validos=25 | rechazados=5
8   2026-10-12T20:19:07 | entrada=solicitudes.txt | total=30 | validos=25 | rechazados=5
9   2026-10-12T20:20:06 | entrada=solicitudes.txt | total=30 | validos=25 | rechazados=5
10  2026-10-12T20:20:26 | entrada=solicitudes.txt | total=30 | validos=25 | rechazados=5
11  2026-10-12T20:25:07 | entrada=solicitudes.txt | total=30 | validos=25 | rechazados=5
12  2026-10-12T20:33:51 | entrada=solicitudes_copia_prueba.txt | total=32 | validos=25 | rechazados=7
13  2026-10-12T20:33:59 | entrada=solicitudes_copia_prueba.txt | total=32 | validos=25 | rechazados=7
14  2026-10-12T20:44:43 | entrada=solicitudes_copia_prueba.txt | total=32 | validos=24 | rechazados=8
```

Etapa 5 Discusión a nivel de maestría

En un informe de 2 a 3 páginas, respondan: ¿cómo cambia la distribución si cuentan las etiquetas sin normalizar? ¿Qué sesgo podría introducir eliminar todos los rechazos? ¿Por qué la tasa de aceptación no demuestra exactitud de los datos? ¿Qué información se pierde con cada transformación? ¿Qué cambiarían si una descripción contuviera | o si el archivo superara la memoria disponible? Sustenten sus respuestas con resultados del ejercicio.

Entregables y evaluación

Entreguen un ZIP con su código completo, la carpeta salidas y el informe. Incluyan nombres, instrucciones para reproducir la ejecución, evidencia de las pruebas y las decisiones de normalización. Si usan IA, indiquen qué partes apoyó y cómo verificaron el código.

Criterio	Puntos
Manipulación de cadenas y justificación de normalización	20
Validación, duplicados y trazabilidad de rechazos	25
Lectura incremental, persistencia y modos de apertura correctos	25
Pruebas, controles y reproducción de resultados	15
Interpretación crítica y claridad del informe	15

Para obtener el puntaje completo, cada criterio debe funcionar y contar con evidencia. Un resultado parcialmente correcto recibe una valoración proporcional; la ausencia de implementación o evidencia recibe cero en el aspecto correspondiente.

Material base: Unidad 03 Cadenas de texto, diapositivas 2–11, y Unidad 04 Archivos y persistencia, diapositivas 2–11, presentaciones proporcionadas por el docente. JSON y JSONL se incorporan como extensión práctica de persistencia.